

XỬ LÝ NGÔN NGỮ TỰ NHIÊN

Xây dựng công cụ tìm kiếm văn bản tương tự

Semantic retrieval với SFT-BE Bi-Encoder, Cross-Encoder reranking và LLM verifier có citation.

Bi-Encoder

Teacher-student distillation

Cross-Encoder reranking

Document retrieval

Bối cảnh bài toán

Tìm kiếm từ khóa

- Phụ thuộc trùng khớp bề mặt.
- Dễ bỏ sót diễn đạt đồng nghĩa.
- Khó xử lý câu hỏi tự nhiên.

Tìm kiếm ngữ nghĩa

- Ánh xạ query và văn bản vào embedding space.
- Tìm đoạn gần nghĩa bằng cosine similarity.
- Có thể kết hợp reranking để tăng precision.

$$\text{sim}(q, d_i) = \cos(f_{\theta}(q), f_{\theta}(d_i))$$

Mục tiêu đề tài

- Xây dựng SFT-BE bằng PyTorch.
- Encode văn bản thành embedding 768 chiều.
- Huấn luyện bằng teacher-student distillation.
- Lập chỉ mục tài liệu PDF thành vector store.
- Truy vấn bằng semantic search.
- Rerank bằng Cross-Encoder.
- Tùy chọn LLM verifier và answer generation.
- Trả kết quả có citation/context window.

768

chiều embedding

6

Transformer layers

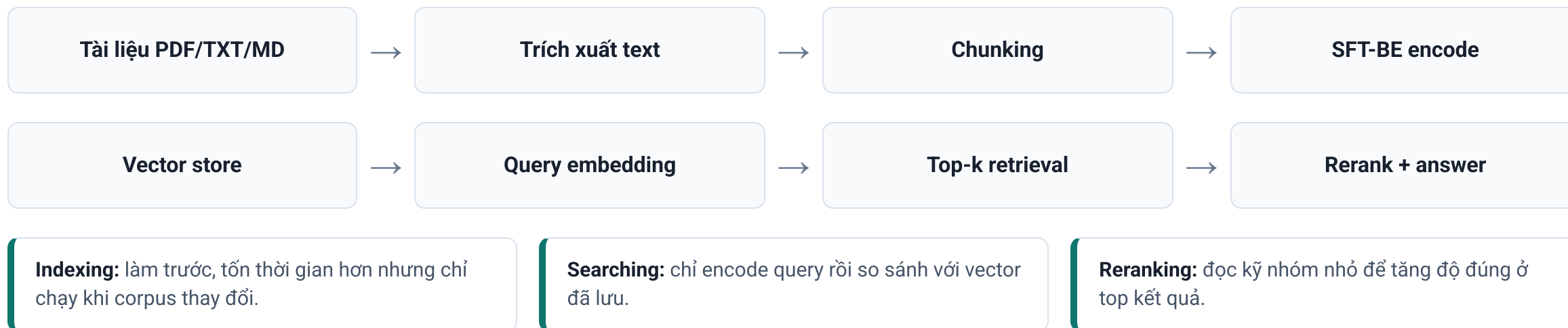
30

paper kiểm thử

3,402

document chunks

Luồng hệ thống tổng thể



$$\mathcal{R}_{final} = \text{Rerank}(\text{TopK}_{c_i \in \mathcal{C}} \cos(z_q, z_i), q)$$

Semantic search

- Query và chunk được đưa vào cùng một không gian vector.
- Hai đoạn gần nghĩa có vector gần hướng nhau.
- Cosine similarity dùng để lấy candidate ban đầu.

$$z_q = f_\theta(q), \quad z_i = f_\theta(d_i)$$

$$\cos(z_q, z_i) = \frac{z_q^\top z_i}{\|z_q\|_2 \|z_i\|_2}$$

Thực tế: hệ thống không hỏi “có trùng từ không”, mà hỏi “hai vector có cùng hướng nghĩa không”.

Giới hạn: embedding có thể làm mờ phủ định, con số hoặc quan hệ chủ thể - đối tượng.

Sparse vector và dense vector

Hướng biểu diễn	Ưu điểm	Hạn chế	Vai trò trong project
Sparse TF-IDF/BM25	Dễ giải thích, tốt với từ khóa hiếm	Khó hiểu đồng nghĩa và ngữ cảnh	Tín hiệu phụ qua lexical boost
Dense Embedding	Mã hóa quan hệ ngữ nghĩa	Khó giải thích từng chiều	Tăng retrieval chính bằng SFT-BE

Vì sao kết hợp? Dense vector bắt nghĩa rộng, sparse signal giữ lại các token đặc biệt như mã paper hoặc tên riêng.

Trong pipeline: lexical boost chỉ là tín hiệu phụ, không thay thế semantic retrieval.

$$s^{base} = 0.82 s^{vec} + 0.18 s^{lex}$$

Bi-Encoder: ý tưởng cốt lõi

Encode độc lập

$$z_q = f_{\theta}(q), \quad z_d = f_{\theta}(d)$$

Query và document không tương tác token-level trong lúc encode.

So sánh vector

$$\text{score}(q, d) = \cos(z_q, z_d)$$

Document embedding có thể precompute trước.

Lợi thế chính: mọi document embedding được tính trước nên query mới xử lý nhanh.

Vai trò: không cố đọc hiểu sâu; nhiệm vụ là lấy candidate đủ tốt cho reranker.

SFT-BE: kiến trúc tổng thể

Tokenizer	`bert-base-uncased`
Max length	128
Factorized dim	128
Hidden size	768
Layers / heads	6 / 12
FFN size	3072
Pooling	Mean pooling + mask

Luồng xử lý đầy đủ

$$x_{ids}, m \rightarrow E(x) \rightarrow H^{(0)} \rightarrow H^{(1)} \rightarrow \dots \rightarrow H^{(6)}$$

$$H^{(6)} \rightarrow \text{MeanPool}(H^{(6)}, m) \rightarrow z \in \mathbb{R}^{768}$$

SFT-BE là encoder-only: nhiệm vụ là nén văn bản thành vector, không sinh chuỗi như decoder.

Điểm thiết kế: giữ hidden size 768 để học trực tiếp từ teacher `all-mpnet-base-v2`, nhưng giảm độ sâu xuống 6 layer để nhẹ hơn.

Tokenization và attention mask

Từ văn bản sang tensor

$$x = (w_1, w_2, \dots, w_L) \rightarrow (t_1, t_2, \dots, t_{128})$$

$$x_{ids} \in \mathbb{N}^{B \times 128}, \quad m \in \{0, 1\}^{B \times 128}$$

Câu ngắn được padding, câu dài bị truncate về 128 token để batch có shape cố định.

Mask cho token thật

$$m_i = \begin{cases} 1 & \text{token thật} \\ 0 & \text{padding} \end{cases}$$

$$A_{ij} = \begin{cases} \frac{q_i^\top k_j}{\sqrt{d_k}} & m_j = 1 \\ -\infty & m_j = 0 \end{cases}$$

Mask xuất hiện trong self-attention và mean pooling để padding không ảnh hưởng embedding.

Ý nghĩa thực tế: nếu không dùng mask, padding token sẽ tham gia attention/pooling và làm lệch embedding của câu ngắn.

Factorized embedding

Embedding trực tiếp

$$W_{direct} \in \mathbb{R}^{30522 \times 768}$$

$$30522 \times 768 \approx 23.4M$$

Nếu dùng trực tiếp, tầng embedding chiếm rất nhiều tham số.

Embedding phân rã

$$e_i = W_E[t_i], \quad h_i^{(0)} = e_i W_P$$

$$W_E \in \mathbb{R}^{30522 \times 128}, \quad W_P \in \mathbb{R}^{128 \times 768}$$

$$30522 \times 128 + 128 \times 768 \approx 4.0M$$

Ý nghĩa: giảm tham số đầu vào từ khoảng 23.4M xuống 4.0M, nhưng representation bên trong vẫn đủ rộng để học ngữ cảnh.

Sinusoidal positional encoding

- Self-attention nhìn các token như một tập nếu không có vị trí.
- Positional encoding đưa thứ tự token vào representation.
- SFT-BE dùng sin/cos nên không tăng tham số học được.

$$PE(pos, 2i) = \sin \left(\frac{pos}{10000^{2i/d_{model}}} \right)$$

$$PE(pos, 2i + 1) = \cos \left(\frac{pos}{10000^{2i/d_{model}}} \right)$$

$$\tilde{h}_i^{(0)} = h_i^{(0)} + PE(i)$$

$$x_i^{(0)} = \text{Dropout}(\text{LayerNorm}(\tilde{h}_i^{(0)}))$$

Vai trò: thứ tự từ rất quan trọng. “the cat bites the dog” và “the dog bites the cat” có cùng token nhưng khác nghĩa.

Scaled dot-product attention

$$X \in \mathbb{R}^{B \times L \times 768}$$

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

$$A = \frac{QK^\top}{\sqrt{d_k}} + M$$

$$\text{Attention}(Q, K, V) = \text{softmax}(A)V$$

- Q
hỏi token cần thông tin gì.
- K
cho biết token khác chứa tín hiệu gì.
- V
là nội dung được trộn theo trọng số attention.
- M
chặn padding bằng giá trị rất âm.

Trực giác: mỗi token chọn những token khác cần “nghe” để cập nhật biểu diễn theo ngữ cảnh, thay vì đọc câu tuần tự như RNN.

Multi-head attention

- SFT-BE dùng 12 attention heads.
- Mỗi head học một kiểu quan hệ token khác nhau.
- Hidden size 768 được chia đều cho các head.

$$d_k = \frac{768}{12} = 64$$

$$\text{head}_r = \text{Attention}(XW_Q^{(r)}, XW_K^{(r)}, XW_V^{(r)})$$

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_{12})W_O$$

Ý nghĩa: nhiều head cho phép mô hình học song song nhiều quan hệ: vị trí gần, từ khóa nội dung, quan hệ cú pháp hoặc tín hiệu ngữ nghĩa.

FFN, GELU và Pre-LN block

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2$$

$$W_1 \in \mathbb{R}^{768 \times 3072}, \quad W_2 \in \mathbb{R}^{3072 \times 768}$$

$$\text{GELU}(x) = x\Phi(x)$$

$$\hat{X}^{(\ell)} = \text{LayerNorm}(X^{(\ell)})$$

$$Y^{(\ell)} = X^{(\ell)} + \text{Dropout}(\text{MHA}(\hat{X}^{(\ell)}))$$

$$X^{(\ell+1)} = Y^{(\ell)} + \text{Dropout}(\text{FFN}(\text{LayerNorm}(Y^{(\ell)})))$$

FFN: biến đổi từng token sau khi token đã thu thập ngữ cảnh từ attention.

Pre-LN: đặt LayerNorm trước sub-layer, giúp gradient ổn định hơn khi train từ đầu.

Mean pooling có attention mask

- Transformer trả hidden state cho từng token.
- Retrieval cần một vector duy nhất cho toàn câu/chunk.
- Mean pooling tận dụng toàn bộ token thật, không chỉ `[CLS]`.

$$H = (h_1, \dots, h_L), \quad h_i \in \mathbb{R}^{768}$$

$$z = \frac{\sum_{i=1}^L m_i h_i}{\sum_{i=1}^L m_i}$$

$$z_k = \frac{\sum_{i=1}^L m_i h_{i,k}}{\sum_{i=1}^L m_i}, \quad k = 1, \dots, 768$$

Output cuối:

$$z = f_{\theta}(x) \in \mathbb{R}^{768}$$

. Vector này là đại diện cho toàn bộ câu/chunk trong retrieval.

Teacher-student distillation

Teacher

$T(\cdot) = \text{all-mpnet-base-v2}$

$$t_i = T(x_i), \quad t_i \in \mathbb{R}^{768}$$

Teacher chỉ sinh target embedding, không update tham số.

Student

$S_\theta(\cdot) = \text{SFT-BE}$

$$s_i = S_\theta(x_i), \quad s_i \in \mathbb{R}^{768}$$

Student nhận gradient để bắt chước embedding space của teacher.

Điểm quan trọng: mô hình học từ embedding mục tiêu, không cần label class hay cặp câu được gán nhãn thủ công.

Cosine distillation loss

$$\hat{s}_i = \frac{s_i}{\|s_i\|_2}, \quad \hat{t}_i = \frac{t_i}{\|t_i\|_2}$$

$$\cos(s_i, t_i) = \hat{s}_i^\top \hat{t}_i$$

$$\mathcal{L}_i = 1 - \cos(s_i, t_i)$$

$$\mathcal{L}_{batch} = 1 - \frac{1}{B} \sum_{i=1}^B \hat{s}_i^\top \hat{t}_i$$

- Loss thấp nghĩa là hướng vector student gần teacher.
- Không cần nhân thủ công cho cặp query-document.
- Phù hợp để pretrain sentence/document embedding.

Cách đọc loss: nếu loss = 0.0736 thì cosine xấp xỉ 0.9264; nghĩa là student đã gần hướng vector của teacher.

Dữ liệu huấn luyện

Wikipedia sentence-level

- `wikimedia/wikipedia, 20231101.en`
- Lọc câu từ 20 đến 500 ký tự
- Tokenize bằng `bert-base-uncased`

Quy mô Stage 0

- 90,250,685 sentence sau xử lý
- 88,445,671 train samples
- 1,805,014 validation samples

$$20 \leq \text{len}(\textit{sentence}) \leq 500$$

Lý do lọc: câu quá ngắn thiếu ngữ cảnh, câu quá dài dễ bị truncate mạnh ở giới hạn 128 token.

Tối ưu hóa khi huấn luyện

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right)$$

$$\eta_t = \eta_{\max} \frac{t}{T_w}, \quad t \leq T_w$$

$$\eta_t = \frac{1}{2} \eta_{\max} \left(1 + \cos \left(\pi \frac{t - T_w}{T - T_w} \right) \right)$$

$$B_{eff} = B \times K = 16 \times 8 = 128$$

Tóm lại: AdamW kiểm soát cập nhật, warmup tránh sốc gradient ban đầu, cosine schedule giảm LR mượt và accumulation mô phỏng batch lớn.

Cross-Encoder reranking

- Dùng pretrained `cross-encoder/ms-marco-MiniLM-L6-v2`.
- Không huấn luyện lại trong đề tài.
- Đọc query và passage trong cùng một sequence.
- Phù hợp để rerank nhóm candidate nhỏ.

$$x = [\text{CLS}], q_1, \dots, q_m, [\text{SEP}], p_1, \dots, p_n, [\text{SEP}]$$

$$x_{ids}, x_{seg}, m$$

$$score(q, p) = g_{\phi}([q; p])$$

Khác với Bi-Encoder: query token có thể attention trực tiếp tới passage token, nên mô hình phân biệt tốt hơn các chi tiết như phủ định, số liệu và entity.

Cấu hình Cross-Encoder

Model	`cross-encoder/ms-marco-MiniLM-L6-v2`	Architecture	`BertForSequenceClassification`
Hidden size	384	Layers	6
Attention heads	12	Head dim	32
Intermediate size	1536	Max position	512
Type vocab	2	Output	1 relevance score

Đánh đổi: Cross-Encoder chính xác hơn ở từng cặp query-passage, nhưng phải forward theo từng candidate nên chỉ dùng sau bước lọc rộng.

Cross-Encoder: embedding và score

$$s_i = \begin{cases} 0 & \text{query/special trước passage} \\ 1 & \text{passage} \end{cases}$$

$$h_i^{(0)} = E_{tok}(t_i) + E_{pos}(i) + E_{seg}(s_i)$$

$$E_{tok} \in \mathbb{R}^{30522 \times 384}, \quad E_{seg} \in \mathbb{R}^{2 \times 384}$$

$$u = \tanh(W_{pool}h_{[CLS]} + b_{pool})$$

$$r = W_{cls}u + b_{cls}$$

$$s^{cross} \in (0, 1), \quad s^{final} = s^{cross}$$

Cách dùng trong hệ thống: score không cần diễn giải như xác suất tuyệt đối; nó chủ yếu là điểm relevance tương đối để sắp xếp các candidate.

Bi-Encoder vs Cross-Encoder

Tiêu chí	Bi-Encoder	Cross-Encoder
Input	Encode query/document độc lập	Ghép query-passage thành một sequence
Tương tác token	Không có trước cosine	Có trong từng attention layer
Tốc độ	Nhanh, precompute được	Chậm hơn, chạy theo từng cặp
Vai trò	First-stage retriever	Reranker top-k

SFT-BE top 100 → Cross-Encoder top 10 → optional LLM top 8 → final top 5

Chiến lược: lấy rộng bằng mô hình nhanh, đọc kỹ bằng mô hình chính xác.

Xây dựng chỉ mục vector

- PDF được trích xuất text theo trang.
- Text được chuẩn hóa và chia chunk.
- Mỗi chunk lưu embedding và metadata.

$$z_i = f_{\theta}(c_i), \quad \hat{z}_i = \frac{z_i}{\|z_i\|_2}$$

$$Z = \begin{bmatrix} \hat{z}_1^T \\ \hat{z}_2^T \\ \vdots \\ \hat{z}_N^T \end{bmatrix} \in \mathbb{R}^{N \times 768}$$

Metadata: paper, section, page và chunk index không tham gia trực tiếp vào cosine, nhưng quyết định khả năng trích dẫn và kiểm chứng nguồn.

Dataset paper dùng cho retrieval

Chủ đề	Số PDF
culture	5
linguistics	5
literature	5
politics	5
psychology	5
sociology	5

30

PDF học thuật

3,402

chunks

768

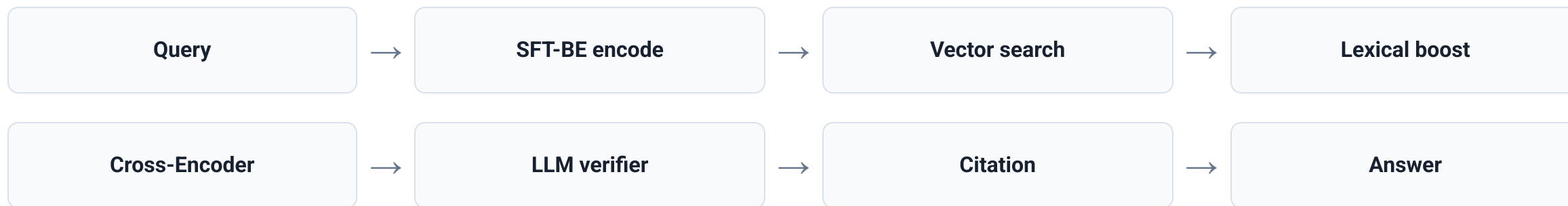
vector dim

46.58s

thời gian build index

Vai trò dataset paper: tập này không dùng để train SFT-BE; nó dùng để kiểm thử retrieval trên tài liệu học thuật thực tế.

Pipeline truy vấn



$$s_i^{base} = 0.82s_i^{vec} + 0.18s_i^{lex}, \quad s_i^{final} = 0.25s_i^{cross} + 0.75s_i^{llm}$$

Fallback: nếu LLM lỗi, hệ thống vẫn trả kết quả sau Cross-Encoder.

LLM verifier và answer generation

Verifier

- Chấm lại evidence theo query.
- Chú ý phủ định, số liệu, entity.
- Fallback nếu LLM lỗi hoặc parse lỗi.

Answer

- Chỉ trả lời dựa trên retrieved context.
- Mỗi claim cần citation.
- Nói rõ nếu bằng chứng chưa đủ.

$$A = \text{LLM}(q, \mathcal{E})$$

Nguyên tắc: LLM chỉ diễn đạt từ evidence đã truy xuất; nếu evidence thiếu thì phải nói thiếu, không tự bịa.

Kết quả huấn luyện Stage 0

Mốc	Global step	Thời gian	Learning rate	Interval loss	Cosine xấp xỉ
Đầu train	500	0.02h	6.36e-7	0.8575	0.1425
25% log	678,500	10.28h	4.92e-4	0.1334	0.8666
50% log	1,356,500	20.71h	4.12e-4	0.0950	0.9050
75% log	2,034,000	32.14h	2.75e-4	0.0819	0.9181
Cuối train	2,712,000	42.39h	1.29e-4	0.0736	0.9264

$$\mathcal{L} = 1 - \cos(student, teacher)$$

Xu hướng: loss giảm rất nhanh ở giai đoạn đầu, sau đó giảm chậm hơn khi student đã gần teacher embedding space.

Đánh giá trên 10MB cuối Wikipedia

47,959

câu sau lọc

0.06567

test loss

0.93433

cosine trung bình

802.78s

thời gian test

Ý nghĩa

Student giữ được hướng embedding gần teacher trên tail-slice Wikipedia.

Không thổi phồng

Đây là đánh giá distillation, không phải accuracy QA hay reasoning.

Cách hiểu đúng: cosine 0.93433 là độ gần với teacher embedding, không phải tỉ lệ trả lời đúng câu hỏi.

Kiểm thử retrieval và reranking

Test logic

- Cross-Encoder sort theo cross score.
- Passage rerank chứa metadata + chunk text.
- Query rỗng bị từ chối.
- Pipeline trả final top 5.

Test truy vấn thực tế

- Câu hỏi cụ thể trên paper `2606.07245`.
- Retrieval trả đúng paper/page.
- Câu gài sai được hệ thống phản bác.

Ý nghĩa kiểm thử: các test xác nhận luồng kỹ thuật chạy đúng; benchmark định lượng retrieval vẫn cần bộ query có nhãn riêng.

Vai trò các thành phần

Thành phần	Ưu điểm	Hạn chế	Vai trò
SFT-BE	Nhanh, precompute được	Không tương tác token-level	First-stage retriever
Lexical boost	Giữ tên riêng, mã paper, con số	Không hiểu đồng nghĩa	Bổ sung vector score
Cross-Encoder	Đọc kỹ query-passage	Chậm hơn	Reranking top-k
LLM verifier	Kiểm tra bằng chứng	Phụ thuộc Ollama	Verification tùy chọn
Vector store	Đơn giản, kiểm soát được	Chưa tối ưu triệu vector	Lưu và truy xuất embedding

Tinh thần thiết kế: mỗi thành phần xử lý một điểm yếu của tầng trước, thay vì kỳ vọng một mô hình duy nhất làm tốt mọi việc.

Phân tích lỗi

PDF extraction

Layout nhiều cột, bảng hoặc text layer lỗi làm sai thứ tự văn bản.

Chunking

Có thể cắt ngang định nghĩa hoặc lập luận dài.

Embedding

Stage 0 học từ Wikipedia, chưa tối ưu mọi paper khoa học.

Query quá chung

Một từ như “politics” không đủ ý định truy vấn.

Sự khác biệt quan trọng: similarity không luôn đồng nghĩa với relevance; cùng chủ đề chưa chắc trả lời đúng câu hỏi.

Hạn chế hiện tại

- Chưa đọc trực tiếp DOCX trong retrieval pipeline.
- Chưa có OCR cho PDF scan.
- Chưa có benchmark retrieval có nhãn cho 30 paper.
- Chunking còn đơn giản với layout phức tạp.
- LLM verifier phụ thuộc Ollama/model local.
- Chưa hỗ trợ multi-hop retrieval trên nhiều vùng xa nhau.

Cách trình bày trung thực: hệ thống đã chạy được end-to-end, nhưng chưa nên báo Precision@k/MRR khi chưa có nhãn retrieval.

Hướng phát triển

Ingestion tốt hơn

DOCX, HTML, OCR, giữ heading/table metadata.

Benchmark riêng

Query có nhãn để đo Precision@k, Recall@k, MRR, nDCG.

Fine-tune domain

Title-abstract, citation context, hard negatives.

Vector database

FAISS, Chroma hoặc Milvus cho corpus lớn.

Hướng quan trọng nhất: xây dựng benchmark có nhãn để so sánh vector-only, vector + lexical, Cross-Encoder và LLM verifier.

Kết luận

Hoàn chỉnh pipeline

Từ train embedding đến retrieval ứng dụng.

Kết quả khả thi

Cosine trung bình 0.93433 với teacher trên tail Wikipedia.

Có thể kiểm chứng

Trả kết quả kèm citation và context window.

SFT-BE đảm nhiệm truy xuất nhanh, Cross-Encoder tăng chất lượng xếp hạng, LLM verifier hỗ trợ kiểm tra bằng chứng khi cần.